字符串算法复习：KMP · Z 函数 · Trie · AC 自动机

> **来源**：Codex 讨论整理（XCPC 复习用） **适用**：单/多模式匹配、前缀与循环节、字典树前缀查询、AC 自动机多串统计、回文；均为确定性算法，与「字符串哈希」互为补充 **关联**：CCPC2025 网络赛 D（哈希 / KMP 二选一）；另见《字符串哈希——双模、梅森质数与随机基数》

〇、总览：问题 → 算法

问题	首选算法	复杂度
单模式 p 在 t 中的所有出现位置	KMP	O(n+m)
串自身的后缀与整体 LCP / 压缩	Z 函数	O(n)
t 每个后缀与 p 的 LCP（扩展 KMP）	Z 构造法 `p + '#' + t`	O(n+m)
单词存在性 / 前缀计数 / 字典序	Trie	O(Σ|p|·26)
多模式串同时匹配一个文本	AC 自动机	O(Σ|p|+n)
最长回文子串 / 回文子串数量	Manacher	O(n)
本质不同子串、字典序第 k 小	SAM / 后缀数组	O(n)
判子串相等（概率方法）	双模 + 随机基数哈希	O(1) 查询

定位：KMP/Z/AC/Manacher 是确定性算法，「必对但模板多」；哈希灵活、模板少但概率正确。需要「出现位置/次数」这类结构信息优先 KMP/AC；只需要「子串等不等」优先哈希。

一、KMP（前缀函数）

定义 pi[i] = s[0..i] 的最长真 border，即最长 l<i+1 使前缀 s[0..l) == 后缀 s[i-l+1..i+1)，pi[0]=0。

思路：失配时跳到次长 border（pi[j-1]），避免回溯 → 均摊 O(1)。

```
vector<int> prefixFunction(const string& s) {
    int n = (int)s.size();
    vector<int> pi(n);
    for (int i = 1; i < n; i++) {
        int j = pi[i - 1];
        while (j > 0 && s[i] != s[j]) j = pi[j - 1];   // 沿 border 链回退
        if (s[i] == s[j]) j++;
        pi[i] = j;
    }

    return pi;
```

```
}

// 返回 p 在 t 中所有出现位置（0 基）
vector<int> kmpMatch(const string& t, const string& p) {
    vector<int> pi = prefixFunction(p), pos;
    for (int i = 0, j = 0; i < (int)t.size(); i++) {
        while (j > 0 && t[i] != p[j]) j = pi[j - 1];
        if (t[i] == p[j]) j++;
        if (j == (int)p.size()) {
            pos.push_back(i - j + 1);
            j = pi[j - 1];            // 允许重叠匹配
        }
    }
    return pos;
}
```

border / 循环节：

- 最小循环节长度 cycle = n − pi[n-1]；若 n % cycle == 0，s 恰好由该循环节整数次重复（如 "abcabc"）。
- 若 n % cycle != 0，则不存在整除 n 的循环节，但 cycle 仍是「最短周期长度」。
- 沿 j = pi[j-1] 一直跳可枚举出 s 的所有 border（常与 DP/栈题配合）。

注意：多组数据记得重置数组；匹配完成用 j = pi[j-1] 继续找重叠出现。

二、Z 函数（Z-algorithm，= 扩展 KMP 的简洁写法）

定义 z[i] = s 与 s[i..] 的最长公共前缀长度（i>0），约定 z[0] = n。

```
vector<int> zFunction(const string& s) {
    int n = (int)s.size();
    vector<int> z(n);
    for (int i = 1, l = 0, r = 0; i < n; i++) {      // [l, r]：已算过的最靠右匹配区间
        if (i <= r) z[i] = min(r - i + 1, z[i - l]); // 继承镜像位置，防止重复比较
        while (i + z[i] < n && s[z[i]] == s[i + z[i]]) z[i]++;
        if (i + z[i] - 1 > r) l = i, r = i + z[i] - 1;
    }
    z[0] = n;
    return z;
}
```

匹配与「扩展 KMP」：构造 c = p + '#' + t（'#' 必须不在文本中），则 z[m+1+i] == m ⟺ p 出现在 t 的 i 处；同时 z[m+1+i] 就是 t[i..] 与 p 的 LCP（被中间的 '#' 截断，天然 ≤ m）。传统扩展 KMP 需维护两套数组，用这个构造法一次 Z 即可等价完成。

每个前缀作为子串的出现次数（含自身位置 0）：

```
// z 已求出。z[i] >= len 表示前缀 s[0..len) 在位置 i 也出现
vector<int> d(n + 1, 0);
for (int i = 1; i < n; i++) if (z[i]) { d[1]++; d[z[i] + 1]--; }  // 差分区间 [1, z[i]]
for (int i = 1; i <= n; i++) d[i] += d[i - 1];
// 前缀长度 len 的出现次数 = 1 + d[len]
```

其他应用：判断 s 是否由某前缀循环重复构成（z[i] == n-i 的点即周期起点）、字符串压缩、（与 KMP 二选一）求循环节。

注意：中间的分隔符 '#' 必须保证不出现在两个串里；字符集大时用 vector 拼接数值代替字符分隔。

三、Trie（字典树）

结构：根为空，边标字符；插入/查询都 O(|s|)。节点数 ≤ Σ|模式串长度|，数组写法空间开 总长 × 字符集。

```
const int MAXN = 1000005;        // ≥ Σ|模式串| + 5
int nxt[MAXN][26], pass[MAXN], ed[MAXN], tot;   // tot 从 0 开始

void insert(const string& s) {
    int u = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (!nxt[u][c]) nxt[u][c] = ++tot;
        u = nxt[u][c];
        pass[u]++;               // 经过次数 = 以该前缀开头的单词数
    }
    ed[u]++;                     // 以该节点结尾的单词数
}

bool findWord(const string& s) {   // 单词是否存在
    int u = 0;
    for (char ch : s) {
        if (!nxt[u][ch - 'a']) return false;
        u = nxt[u][ch - 'a'];
    }
    return ed[u] > 0;
}

int countPrefix(const string& s) { // 有多少个单词以 s 为前缀
    int u = 0;
    for (char ch : s) {
        if (!nxt[u][ch - 'a']) return 0;
        u = nxt[u][ch - 'a'];
    }
    return pass[u];
}
```

01-Trie（异或最大值，高频套路）：

```
int ch[MAXN * 31][2], tot;   // MAXN: 数字个数上限，节点数 ≈ n·31
void insertXor(int x) {
    int u = 0;
    for (int b = 30; b >= 0; b--) {
        int c = (x >> b) & 1;
        if (!ch[u][c]) ch[u][c] = ++tot;
        u = ch[u][c];
    }
}
int maxXor(int x) {                 // 集合中与 x 异或的最大值
    int u = 0, res = 0;
    for (int b = 30; b >= 0; b--) {
        int c = (x >> b) & 1;
        if (ch[u][c ^ 1]) { res |= 1 << b; u = ch[u][c ^ 1]; }
        else              { u = ch[u][c]; }
    }
    return res;
}
```

注意：字符集不是 26 时改用 unordered_map 子节点 / 指针写法 / 离线离散化；01-Trie 数组要开 点数×位数。

四、AC 自动机（多模式匹配）

本质：Trie + fail 指针。fail[u] 指向「u 代表的串的最长真后缀中、也存在于 Trie 里的那个串的节点」；BFS 层序构建。文本匹配时走到节点 u，u 沿 fail 链到根的所有节点都命中。

补边技巧：构建时若 nxt[u][c] 不存在，就补成 nxt[fail[u]][c]（把 Trie 补成自动机），查询时 O(1) 直达，不用循环跳 fail。

```
const int MAXN = 500005;          // ≥ Σ|模式串| + 5
int nxt[MAXN][26], fail[MAXN], tot;
vector<int> order, ed;          // order: BFS 序; ed: 模式串终点

void insert(const string& s) {
    int u = 0;
    for (char ch : s) {
        int c = ch - 'a';
        if (!nxt[u][c]) nxt[u][c] = ++tot;
        u = nxt[u][c];
    }
    ed.push_back(u);             // 记录模式串终点编号
}

void build() {
    queue<int> q;
    for (int c = 0; c < 26; c++)
        if (nxt[0][c]) q.push(nxt[0][c]);
    while (!q.empty()) {
        int u = q.front(); q.pop();
        order.push_back(u);
```

```
        for (int c = 0; c < 26; c++) {
            int& v = nxt[u][c];
            if (v) { fail[v] = nxt[fail[u]][c]; q.push(v); }
            else    v = nxt[fail[u]][c];          // 补边，查询免跳链
        }
    }
}

// 文本走一遍，每个访问节点 +1；再沿 fail 树从深到浅累加：
// 节点 u 的累加值 = 该节点代表串的出现次数
vector<int> cnt;                  // 大小 tot+1，初始 0
void countByFailTree(const string& t) {
    int u = 0;
    for (char ch : t) { u = nxt[u][ch - 'a']; cnt[u]++; }
    for (int i = (int)order.size() - 1; i >= 0; i--) {
        int v = order[i];
        cnt[fail[v]] += cnt[v];   // 父是子的「后缀」，子出现则父必出现
    }
}
// 模式串 i 的出现次数 = cnt[ed[i]]
```

只问「文本包含多少个模式」（去重）：走文本时对每条 fail 链上未访问节点累加并打 -1 标记即可；若还要各模式次数，一律用上面的 fail 树累加，不要逐点跳链（最坏会退化）。

fail 树的用途（常考）：

- 统计每个模式串出现次数（上面的子树和）。
- 判「某模式是否为另一模式的后缀」：Trie 节点 u 的祖先关系 ⇔ 子树关系。
- AC 自动机上 DP：统计不含任何模式的串数/串长、字符串生成计数（在自动机上按字符转移，禁止走到模式终点及其 fail 后代）。

注意：build 前必须先 insert 完所有模式；根的儿子先入队；补边后 nxt 已被改写，不能再拿它判断「原始子节点」（需要时另存）。

五、Manacher（回文）

核心：用已算区间的镜像信息继承半径，再中心扩展 → 每个字符均摊 O(1)。

```
// d1[i]：以 i 为中心的最长奇回文半径（长度 = 2*d1[i]-1）
vector<int> manacherOdd(const string& s) {
    int n = (int)s.size();
    vector<int> d1(n);
    for (int i = 0, l = 0, r = -1; i < n; i++) {
        int k = (i > r) ? 1 : min(d1[l + r - i], r - i + 1);
        while (i - k >= 0 && i + k < n && s[i - k] == s[i + k]) k++;
        d1[i] = k--;
        if (i + k > r) l = i - k, r = i + k;
    }
    return d1;
}
```

```cpp
// d2[i]: 中心在 i-1 与 i 之间的偶回文半径（长度 = 2*d2[i]）
vector<int> manacherEven(const string& s) {
    int n = (int)s.size();
    vector<int> d2(n);
    for (int i = 0, l = 0, r = -1; i < n; i++) {
        int k = (i > r) ? 0 : min(d2[l + r - i + 1], r - i + 1);
        while (i - k - 1 >= 0 && i + k < n && s[i - k - 1] == s[i + k]) k++;
        d2[i] = k--;
        if (i + k > r) l = i - k - 1, r = i + k;
    }
    return d2;
}
```

结论:

- 最长回文子串长度 = max(2·d1[i]−1, 2·d2[i])。
- 回文子串总数 = Σd1[i] + Σd2[i]（每个中心贡献其半径个不同长度）。
- 回文半径数组本身常被拿去 DP（回文分割、计数），比哈希+二分更快。

六、实战选择

- 先判断问题要「结果」还是「结构」：只问相等/存在 → 哈希或 Trie；要位置、次数、前缀/后缀关系 → KMP / Z / AC。
- 多模式串统计一律 AC + fail 树；单个模式用 KMP；只要匹配位置、不要求别的，Z 构造法也够。
- 经典组合题型：AC 自动机 + 矩阵快速幂（长串计数）、KMP + 栈（消去匹配段）、Trie + DFS（字典序）、Manacher + DP（回文划分）、01-Trie（异或最值）。
- 空间敏感时优先数组写法并精确开 MAXN；字符集/数据格式特殊（如数字串）用 vector 存序列再套模板。
  *** End Patch